FormBuilder Application

1. Project Task Definition

1.1 Project Title

Configurable Form Builder Platform

1.2 Purpose of the Project

The purpose of creating a form builder is to provide a reusable, configurable mechanism for defining, rendering, validating, and persisting data capture forms without requiring custom backend and frontend development for each new form.

The form builder aims to:

- Eliminate repetitive form-related development effort.
- Enable rapid creation and modification of forms by non-developer users.
- Standardize data capture, validation, and persistence mechanisms.
- Provide operational visibility and manageability of submitted form data.

1.3 Problem Statement

In the absence of a form builder, each new form requires:

- Custom UI development.
- Custom backend validation logic.
- Custom database schema design.
- Custom listing and management screens.

This approach is slow, error-prone, and difficult to maintain. The form builder addresses these issues through configuration-driven form definition and execution.

2. Scope Definition

2.1 In-Scope

- Visual form creation and editing
- Dynamic form rendering
- Configurable validations and conditional logic
- Table-per-form database persistence
- Submission listing and bulk management
- Authentication and authorization

2.2 Explicitly Out of Scope

- Workflow engines
- Analytics and monitoring
- Localization and accessibility features
- PDF generation
- Virus scanning
- Rule engines
- E-signatures
- AI-assisted features
- Documentation deliverables
- Unit and integration testing

3. Functional Requirements

3.1 Form Definition and Visual Editing

- The system shall provide a browser-based visual editor for creating and modifying forms.
- The editor shall support drag-and-drop placement of fields onto a canvas.
- The editor shall allow reordering of fields and grouping them into sections.
- The editor shall support multi-step (wizard-style) forms using page breaks.

3.2 Supported Field Types

- Text (single-line, multi-line)
- Numeric (integer, decimal)
- Date, time, date-time
- Dropdown (single-select, multi-select)
- Radio buttons and checkboxes
- Boolean toggle
- File upload (basic storage)
- Hidden/system fields
- Static elements such as labels and section headers

3.3 Field Configuration

- Each field shall support mandatory/optional configuration.
- Fields shall support default values.
- Fields shall support read-only and disabled states.
- Fields shall support place-holder and help text.
- Fields may exist that are not shown to the end user

3.4 Validation Capabilities

- The system shall allow custom validation rules at the field level.
- Validations may reference values of other fields.
- Conditional validations shall be supported based on form state.
- The system shall support form-level validation that executes once all mandatory fields are populated.
- Submission shall be blocked if validation errors exist, with consolidated error feedback.

3.5 Conditional Logic

- Fields and sections may be shown or hidden based on conditions.
- Fields may be enabled or disabled conditionally.
- Calculated fields may derive values from other inputs.

3.6 Data Persistence

- Each form shall have a dedicated database table.
- Each field shall map to a corresponding table column.
- Submissions shall be timestamped.
- Draft and final submission states shall be supported.
- Schema evolution shall be handled through controlled application-level migrations (no migration tools).

3.7 Submission Management

- The system shall provide a grid view listing all submissions of a form.
- The grid shall support sorting, filtering, and pagination.
- Users shall be able to select multiple entries.
- Bulk operations shall include [soft] delete and other basic operations.
- Bulk export to CSV/xlsx shall be supported.

3.8 Access Control

- The system shall restrict form creation and editing to authorized users.
- Submission viewing and bulk actions shall be role-controlled.
- Session-based authentication shall be enforced using Spring Security.

4. Non-Functional Requirements

4.1 Technology Stack

- FrontEnd NextJS app hosted on Node
- Java version shall be 21.
- The backend framework shall be Spring Boot 3.

- Database shall be PostgreSQL 17.

4.2 Build and Dependency Management

- Maven shall be used as the build tool.
- Maven profiles shall not be used.

4.3 Authentication and Session Management

- NextJS shall use a simplified authentication mechanism suitable only for this project [intentionally non-production-grade].
- Backend authentication shall be session-based.
- JWT shall not be used.

4.4 Concurrency and Runtime Constraints

- Virtual threads shall not be used.
- Traditional thread-per-request model shall be assumed.

4.5 Database Constraints

- Flyway and Liquibase shall not be used.
- Schema changes shall be managed explicitly within application lifecycle logic.

4.6 Frontend Constraints

- JavaScript and jQuery may be integrated where required.
- The system shall be usable in modern desktop browsers.

5. Final Outcome Definition

At the conclusion of the project, the system should exhibit the following characteristics:

- Administrators can visually design and publish forms without code changes.
- End users can fill and submit forms rendered dynamically from configuration.
- Submitted data is persisted in dedicated tables per form.
- Submissions can be viewed, filtered, and managed in grid-based screens.
- Validation logic is enforced consistently across UI and backend.
- The platform operates as a reusable internal capability rather than a one-off solution.